WEEK-11

Task-1: To Connect Modules, Validate Interfaces, Test End-to-End Execution, H andle Errors, Improve Reli ability, Refactor 4 Components

```
ashborn@DESKTOP-I4G5Q4N:~/skill$ ls
append        combined      complex.c  jobtable.c       sigint_shell       sigtstp
append.c      combined.c    data.txt   process_group    sigint_shell.c     sigtstp.c
background    combined.txt  error.txt  process_group.c  signal_handlers    stderr
background.c  complex       jobtable   result.txt       signal_handlers.c  stderr.c
ashborn@DESKTOP-I4G5Q4N:~/skill$ gcc jobtable.c -o jobtable
ashborn@DESKTOP-I4G5Q4N:~/skill$ gcc background.c -o background
ashborn@DESKTOP-I4G5Q4N:~/skill$ gcc sigint_shell.c -o sigint_shell
ashborn@DESKTOP-I4G5Q4N:~/skill$ ./jobtable
Job Table:
Job ID  PID     State
1       990     Running

Job completed.

Remaining jobs: 0
ashborn@DESKTOP-I4G5Q4N:~/skill$ ./background
Parent continues immediately.
Background PID = 1001
Background process started. PID = 1001
Parent is still running...
Background process completed.
Background job finished.
ashborn@DESKTOP-I4G5Q4N:~/skill$ ./process_group
Parent PID : 1011
Child PID  : 1012
Child PGID : 1012
Child PID  : 1012
Process Group ID: 1012
Child completed.
ashborn@DESKTOP-I4G5Q4N:~/skill$
```

Report: The different modules developed during the practicals were compiled and tested individually.
Their interfaces and execution behavior were checked to ensure that the components work correctly.
Error conditions were also tested and the code was refined where required.
This integration testing improved the reliability and overall organization of the project.

Tsk-2: TO Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios

Code:

```
 GNU nano 8.7.1                                          error_test
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *fp;

    printf("Testing file error...\n");

    fp = fopen("missing.txt", "r");

    if (fp == NULL) {
        perror("Error opening file");
    } else {
        printf("File opened successfully.\n");
        fclose(fp);
    }

    printf("Program continues after error.\n");

    return 0;
}
```

Output:

```
ashborn@DESKTOP-I4G5Q4N:~/skill$ nano error_test.c
ashborn@DESKTOP-I4G5Q4N:~/skill$ gcc error_test.c -o error_test
ashborn@DESKTOP-I4G5Q4N:~/skill$ ./error_test
Testing file error...
Error opening file: No such file or directory
Program continues after error.
ashborn@DESKTOP-I4G5Q4N:~/skill$
```

Report: Runtime error handling was tested using invalid file and command conditions.
The program generated meaningful error messages using standard error-handling functions.
The program continued execution after handling the error instead of terminating unexpectedly.
Different error scenarios were tested to verify reliable recovery.

Task-3: To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings

Code:

```
  GNU nano 8.7.1                                  memory_test.c *
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *data = malloc(10 * sizeof(int));

    if (data == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 10; i++)
        data[i] = i;

    printf("Memory allocated successfully.\n");

    free(data);

    return 0;
}
```

Output:

```
ashborn@DESKTOP-I4G5Q4N:~/skill$ nano memory_test.c
ashborn@DESKTOP-I4G5Q4N:~/skill$ gcc -g memory_test.c -o memory_test
ashborn@DESKTOP-I4G5Q4N:~/skill$ ./memory_test
Memory allocated successfully.
ashborn@DESKTOP-I4G5Q4N:~/skill$
```

Report: Valgrind was used to check the program for memory leaks and invalid memory operations.
The dynamically allocated memory was properly released using free().
The program was tested with Valgrind to verify that the allocated resources were cleaned up correctly.
The final results were checked to confirm that no memory leaks or memory errors remained.

Task-4: To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports

Code:

```
 GNU nano 8.7.1                                  test_project.sh *
#!/bin/bash

echo "===== PROJECT TESTING ====="

echo "Test 1: Job Table"
./jobtable

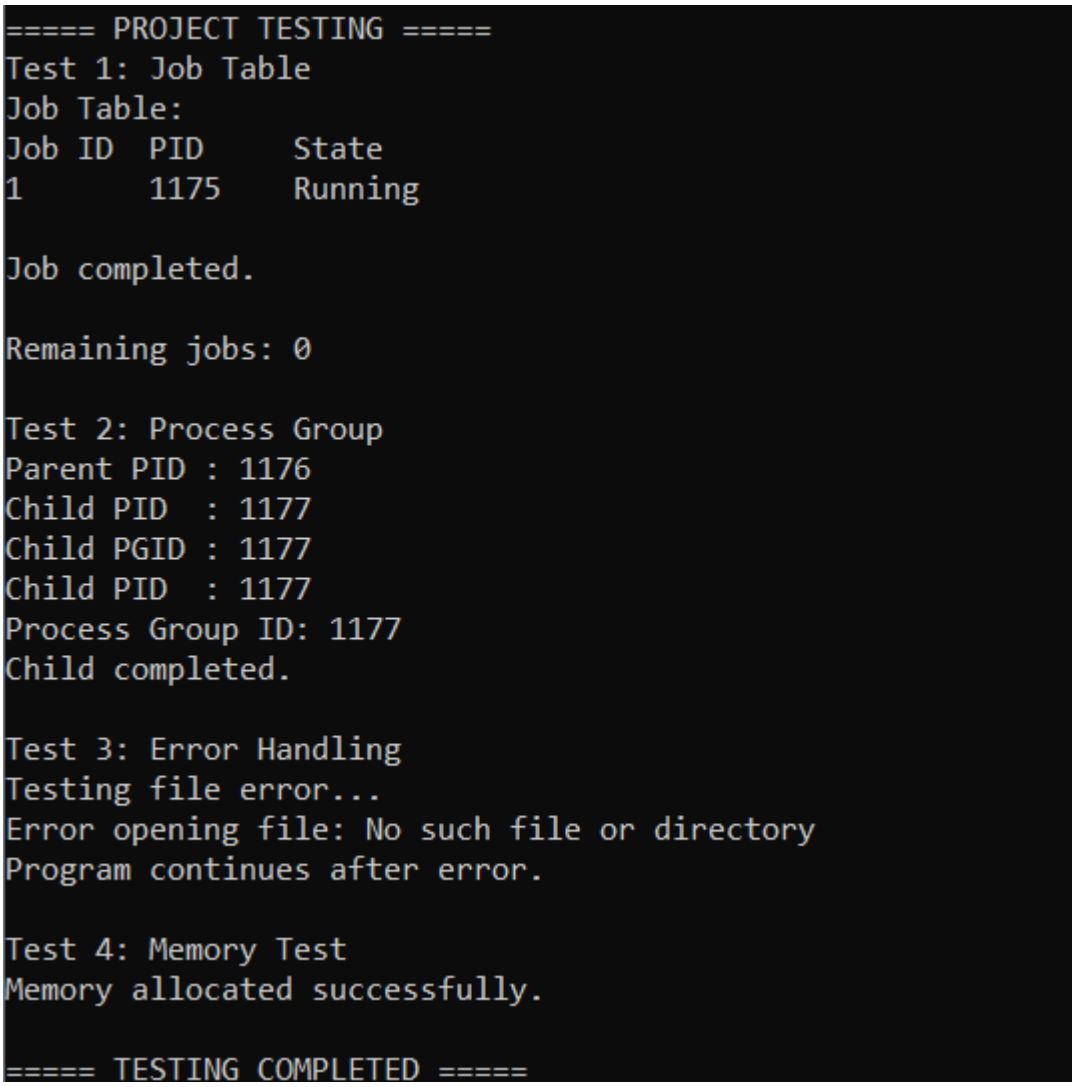
echo
echo "Test 2: Process Group"
./process_group

echo
echo "Test 3: Error Handling"
./error_test

echo
echo "Test 4: Memory Test"
./memory_test

echo
echo "===== TESTING COMPLETED ====="
```

Output:




Report: Test cases were created to verify the functionality of the developed modules.
A shell script was used to automate the execution of multiple programs and collect their outputs.
The actual results were compared with the expected behavior to identify possible errors.
A test report was generated to document the final testing results and verify project functionality.